

编译原理大作业 1 & 2

类 Rust 语言的词法/语法分析、语义分析与四元式中间代码生成

2351078 李堂辉

2351837 高胜寒

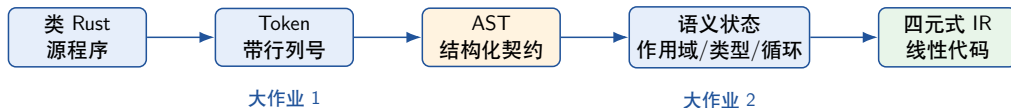
2352197 冉子易

(按学号为序)

同济大学

2026 年 6 月 16 日

主线：两个作业串成一条前端流程



实现重点

这份实现把类 Rust 语法保留为结构化 AST，并在第二个作业中继续完成语义诊断和 IR 生成。

我们把重点放在三条实现线索

线索	代码落点	值得讲的点
语法覆盖	<code>parser.rs</code> , <code>ast.rs</code>	覆盖引用、数组、元组、表达式块和三类循环
语义状态	<code>compiler.rs</code>	作用域栈、可变性、返回类型、循环栈同时参与检查
IR 线性化	<code>ir.rs</code> , <code>compiler.rs</code>	临时变量承接表达式, 标签和跳转承接结构化控制流

源程序先变成 AST, 再完成语义检查, 最后生成四元式 IR。

代码入口：第一份 AST 被第二份继续消费

大作业 1

lexer.rs 负责字符扫描；parser.rs 负责递归下降；ast.rs 定义统一的程序结构。

- 输出 Token 序列和 AST 打印。
- 测试按课程规则编号组织。

大作业 2

继续沿用 Lexer、Parser 和 AST，再增加 compiler.rs 与 ir.rs。

- 语义错误统一进入 errors。
- 合法程序生成四元式 IR。

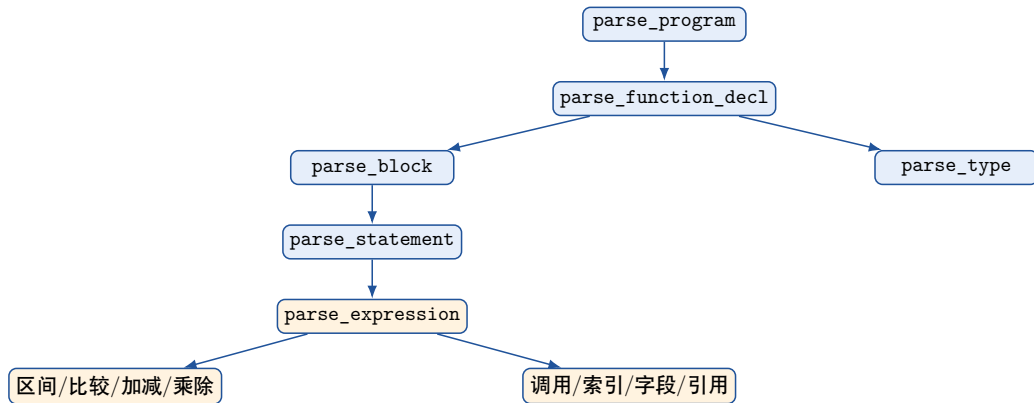
类 Rust 子集的结构覆盖

覆盖范围

类型	i32, &T, &mut T, 数组, 元组
左值	标识符, 解引用, 数组下标, 元组下标
控制流	if, while, for, loop
表达式	调用, 区间, 表达式块, if 表达式

- 核心难点是让这些结构进入同一棵 AST。
- 后续语义阶段可以继续检查类型、作用域和控制流。
- 所以作业 1 的关键产物是“可继续使用的 AST”。

Parser: 把复杂语法压进可维护的层次



这里保留递归下降的优势：结构规则和表达式优先级分开，出错位置也容易落到具体解析函数。

AST: 保留语义和 IR 需要的信息

```
pub enum Type {  
    I32,  
    Ref(Box<Type>),  
    MutRef(Box<Type>),  
    Array(Box<Type>, i64),  
    Tuple(Vec<Type>),  
}
```

```
pub enum Expr {  
    BinaryOp(...),  
    Ref(Box<Expr>),  
    MutRef(Box<Expr>),  
    Index(Box<Expr>, Box<Expr>),  
    TupleIndex(Box<Expr>, i64),  
    ExprBlock(ExprBlock),  
    IfExpr(...),  
}
```

设计取向

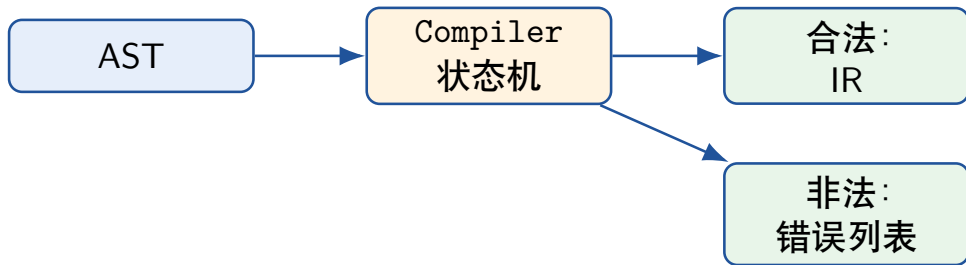
AST 保留类型、左值形态、控制流和表达式形态，给第二个作业直接消费。

作业 1 产出：可继续编译的结构

产物	包含的信息	给作业 2 的价值
Token	类别、字面量、行列号	定位词法/语法问题
AST 语句	声明、赋值、返回、分支、循环	语义检查按语句类型分派
AST 表达式	类型相关结构、调用、索引、引用	推导类型并生成临时变量
AST 类型	五类 Type	赋值、返回、解引用检查的依据

这就是两个大作业能串起来的关键。

作业 2 的转折: 遍历 AST 时带着状态走



状态	代码字段	用途
作用域	scopes	找变量、查类型、查可变性
函数返回	current_return_type	检查 return 是否匹配签名
循环上下文	loop_stack	检查并翻译 break/continue
生成计数	temp_count, label_count	分配 tN 和 LN

符号表栈：作用域、类型、可变性放在一起

```
struct SymbolInfo {  
    mutable: bool,  
    ty: Type,  
    arg: Arg,  
}  
  
struct Compiler {  
    scopes: Vec<HashMap<String, SymbolInfo>>,  
    errors: Vec<String>,  
}
```

- 进入函数、块、表达式块时压栈。
- 退出作用域时弹栈，局部变量自然失效。
- 查找变量时从内层向外层倒序查找。
- 允许 shadowing，和类 Rust 语言直觉一致。

语义检查：围绕 Rust 风格约束

检查点	触发场景	背后的状态
未声明变量	<code>a = 1;</code> 但没有声明	<code>lookup_variable</code>
不可变变量赋值	<code>let a = 1; a = 2;</code>	<code>mutable</code> 字段
类型不匹配	数组赋给 <code>i32</code> , 或 <code>i32 + tuple</code>	Type 枚举
引用/解引用约束	对非引用解引用, 或写不可变引用	<code>Ref/MutRef</code>
返回类型	<code>-> i32</code> 但 <code>return;</code>	当前函数返回类型
循环控制	循环外 <code>break/continue</code>	<code>loop_stack</code>

错误汇总：一次运行看出多类问题

```
fn test() -> i32 {  
    let a: i32 = 1;  
    a = 2;  
    break;  
    return;  
}
```

[Semantic Error] 不可变变量 'a' 不允许被二次赋值
[Semantic Error] **break** 必须出现在循环体内
[Semantic Error] 返回语句类型空和函数声明类型 i32 不一致

实现方式

Compiler 会继续收集语义错误，统一写入 `errors: Vec<String>`，最后集中输出。

IR: 从树形结构变成线性四元式

```
pub enum Arg {  
    Empty,  
    Int(i64),  
    Var(String),  
    Temp(usize),  
    Label(usize),  
}
```

```
pub struct Quadruple {  
    op: Op,  
    arg1: Arg,  
    arg2: Arg,  
    result: Arg,  
}
```

指令族	代表指令
算术/比较	ADD, SUB, MUL, DIV, EQ, NE, LT, LE, GT, GE
控制流	LABEL, JUMP, JUMP_IF_FALSE
调用/返回	ARG, CALL, PARAM, RETURN
引用/聚合	REF, Deref, Deref_Assign, LOAD_ARRAY

表达式翻译：临时变量串起计算

```
let mut c = a + b * 2;
```

```
(MUL,    b,    2,    t0)
(ADD,    a,    t0,    t1)
(ASSIGN, t1,    -,    c)
```

递归翻译表达式时，每个中间结果都拿到一个 t_N 。

`compile_expression` 的返回值是 $(Type, Arg)$ ：既给语义检查，也给 IR 继续使用。

分支翻译：条件失败跳转

```
if a > 0 {  
    return 1;  
} else {  
    return 0;  
}
```

```
(GT,          a, 0, t0)  
(JUMP_IF_FALSE, t0, -, L0)  
(RETURN,      1, -, -)  
(JUMP,        -, -, L1)  
L0:  
(RETURN,      0, -, -)  
L1:
```

结构化的 if/else 被压成“条件、失败标签、结束标签”。

循环翻译：标签和循环栈

入口标签

start_label

- while 重新判断条件。
- loop 直接进入下一轮。
- continue 跳到这里。

出口标签

end_label

- 条件失败跳到这里。
- break 跳到这里。
- 循环体结束后接下一条指令。

```
loop_stack.push((start_label, end_label, None))
```

非基础结构：数组、引用、调用都有落点

源码结构	AST 节点	IR 或语义落点
<code>&x / &mut x</code>	<code>Ref / MutRef</code>	生成 REF, 类型变成引用类型
<code>*p</code>	<code>UnaryDeref</code>	检查引用类型, 生成 Deref
<code>a[i]</code>	<code>Index</code>	检查数组类型, 读取生成 LOAD_ARRAY
<code>t.0</code>	<code>TupleIndex</code>	检查元组类型并取对应元素类型
<code>foo(a,b)</code>	<code>Call</code>	参数生成 ARG, 调用生成 CALL

这些结构承接作业 1 中较完整的类 Rust 子集, 并在 IR 层保留对应的读取、引用、调用和类型检查落点。

测试覆盖：功能集和诊断集分开

测试入口	覆盖内容	观察结果
test_all.rs.txt	函数、声明、表达式、分支、循环、引用、数组、元组	作业 1 输出 Token/AST, 作业 2 输出 IR
test_errors.rs.txt	非法字符、缺分号、括号不匹配等	Lexer/Parser 报错
test_semantic_errors.rs.txt	未声明、类型不匹配、不可变赋值、非法控制流、返回类型错误	[Semantic Error] 列表
cargo check	Rust 工程可编译性	两个正式工程均无编译错误

汇报收束：我们真正完成的是一条前端流水线

一句话总结

这两个作业合起来完成了从类 Rust 源程序到四元式 IR 的前端流程。

- 作业 1 的重点是把复杂语法结构化，尤其是引用、数组、元组、表达式块和循环结构。
- 作业 2 的重点是带状态遍历 AST，把作用域、类型、可变性、返回类型和循环上下文检查清楚。
- 最终 IR 用临时变量和标签把表达式和控制流线性化，为后续解释执行或目标代码生成留下接口。

谢谢老师，欢迎提问

备用：如果问“两份作业区别是什么？”

项目	输入	核心状态	输出
大作业 1	源代码字符串	Token 指针、解析函数栈	Token + AST
大作业 2	AST	符号表栈、循环栈、返回类型、计数器	语义错误或 IR

备用: while 的 IR 生成细节

```
let start_label = self.new_label();  
let end_label = self.new_label();  
  
emit(Label,      -,      -, Label(start_label));  
let cond_arg = compile_expression(cond);  
emit(JumpIfFalse, cond_arg, -, Label(end_label));  
  
loop_stack.push((start_label, end_label, None));  
compile_block(body);  
loop_stack.pop();  
  
emit(Jump,      -,      -, Label(start_label));  
emit(Label,     -,      -, Label(end_label));
```

备用：实现边界

- 当前重点是前端流程和四元式表达，没有继续做目标代码生成。
- 函数调用已经生成 ARG/CALL，但函数签名检查仍可继续完善。
- 数组读取有 LOAD_ARRAY，数组写入接口存在，后续可以扩展更完整的地址计算和 STORE_ARRAY。
- for/range 进入语法树，IR 层当前重点展示 while/loop/break/continue 的标签化控制流。